

1. TEXTURING — Functions, Constants, and Mini-Examples

`gl.createTexture()`

Creates an empty GPU texture object.

```
const tex = gl.createTexture();
```

`gl.bindTexture(target, texture)`

Selects which texture you're modifying.

Allowed targets:

- `gl.TEXTURE_2D`
- `gl.TEXTURE_CUBE_MAP`

```
gl.bindTexture(gl.TEXTURE_2D, tex);
```

After binding, all texture operations affect this texture.

`gl.texImage2D(...)`

Uploads pixel data.

Parameters:

- `target` → `gl.TEXTURE_2D`
- `level` → 0
- `internalFormat` → `gl.RGBA`
- `width, height` OR image object
- `type` → `gl.UNSIGNED_BYTE`

Example (real image):

```
gl.texImage2D(gl.TEXTURE_2D, 0, gl.RGBA,  
             gl.RGBA, gl.UNSIGNED_BYTE, image);
```

Example (1×1 placeholder pixel):

```
gl.texImage2D(gl.TEXTURE_2D, 0, gl.RGBA,  
             1, 1, 0, gl.RGBA, gl.UNSIGNED_BYTE,  
             new Uint8Array([128, 128, 128, 255]));
```

`gl.texParameteri(target, pname, param)`

Controls filtering and wrapping.

Filtering constants:

- `gl.NEAREST`
- `gl.LINEAR`
- `gl.LINEAR_MIPMAP_LINEAR`
- `gl.NEAREST_MIPMAP_NEAREST`

Wrapping constants:

- `gl.REPEAT`
- `gl.CLAMP_TO_EDGE`
- `gl.MIRRORED_REPEAT`

Example:

```
gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_MIN_FILTER, gl.LINEAR_MIPMAP_LINEAR);  
gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_MAG_FILTER, gl.LINEAR);  
gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_WRAP_S, gl.REPEAT);
```

`gl.generateMipmap(gl.TEXTURE_2D)`

Builds smaller texture versions.

```
gl.generateMipmap(gl.TEXTURE_2D);
```

```
gl.activeTexture(gl.TEXTURE0 + i)
```

Selects texture unit.

```
gl.activeTexture(gl.TEXTURE0); // slot 0
```

```
gl.uniform1i(location, unitIndex)
```

Connects sampler2D → texture unit.

```
gl.uniform1i(uDiffuseTex, 0); // use TEXTURE0
```

```
gl.uniform1i(uBumpTex, 1); // use TEXTURE1
```

2. LIGHTING — Functions, Uniforms, Constants, Mini-Examples

Lighting uses uniforms, not many WebGL functions.

```
gl.uniform3f(location, x, y, z)
```

Sets vec3 uniforms: light direction, color, ambient, camera pos.

```
gl.uniform3f(uLightDir, 0.4, -1.0, 0.3);
```

```
gl.uniform3f(uLightColor, 1.0, 1.0, 1.0);
```

```
gl.uniform3f(uAmbientColor, 0.3, 0.3, 0.3);
```

```
gl.uniform3f(uViewPos, camX, camY, camZ);
```

GLSL fragment shader usage:

```
vec3 L = normalize(uLightDir);
```

```
vec3 N = normalize(vNormal);
```

```
float diff = max(dot(N, L), 0.0);
```

reflect(I, N) in GLSL

Used for specular highlight.

```
vec3 R = reflect(-L, N);
```

```
float spec = pow(max(dot(R, V), 0.0), 32.0);
```

3. CAMERA — Matrices Explained With Real Matrix Representations

Your 3D pipeline uses three transformation matrices:

1. Model matrix (object → world)
2. View matrix (world → camera)
3. Projection matrix (camera → clip space)

Each matrix is a 4×4 matrix and multiplied in this order:

```
gl_Position = Projection * View * Model * vec4(position, 1.0);
```

Below are the actual matrix forms with explanations and small example snippets (no utility functions).

MODEL MATRIX (Object Transform)

The model matrix positions, rotates, and scales your models (cube & cat) in the world.

Translation Matrix

Moves object by (tx, ty, tz):

$$M_{\text{translate}} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ t_x & t_y & t_z & 1 \end{bmatrix}$$

Usage:

```
model = [  
  1,0,0,0,  
  0,1,0,0,  
  0,0,1,0,  
  tx,ty,tz,1  
];
```

Scaling Matrix

$$M_{\text{scale}} = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Example (uniform scale):

```
model = [  
  s,0,0,0,  
  0,s,0,0,  
  0,0,s,0,  
  0,0,0,1  
];
```

Rotation Around Y-Axis

$$M_{\text{rotY}} = \begin{bmatrix} \cos \theta & 0 & -\sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ \sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Mini-example:

```
const c = Math.cos(angle);  
const s = Math.sin(angle);  
model = [  
  c,0,-s,0,  
  0,1,0,0,  
  s,0,c,0,  
  0,0,0,1  
];
```

VIEW MATRIX (Camera)

The view matrix moves the entire scene so that the camera behaves like it is at a specific position.

Inputs:

- eye = camera position
 - center = point camera looks at
 - up = usually (0,1,0)
-

Compute Direction Vectors

Forward (Z axis):

$$\vec{z} = \frac{\text{eye} - \text{center}}{\| \text{eye} - \text{center} \|}$$

Right (X axis):

$$\vec{x} = \frac{up \times \vec{z}}{\| up \times \vec{z} \|}$$

Up (Y axis):

$$\vec{y} = \vec{z} \times \vec{x}$$

Form View Matrix

Example (building the matrix manually):

```
V = [
  xx, xy, xz, 0,
  yx, yy, yz, 0,
  zx, zy, zz, 0,
  -(xx*ex + xy*ey + xz*ez),
  -(yx*ex + yy*ey + yz*ez),
  -(zx*ex + zy*ey + zz*ez),
  1
];
```

PROJECTION MATRIX (Perspective)

Converts 3D coordinates into clip space with perspective.

Inputs:

- fov = vertical angle in radians
- aspect = width / height
- near, far = clipping planes

Matrix Formula

Example:

```
const f = 1 / Math.tan(fov/2);
P = [
  f/aspect, 0, 0, 0,
  0, f, 0, 0,
  0, 0, (far+near)/(near-far), -1,
  0, 0, (2*far*near)/(near-far), 0
];
```

4. CAMERA ORBITING — Turning Mouse Input Into a View Matrix

Mouse input updates:

- theta → horizontal rotation
- phi → vertical rotation
- radius → camera distance

Convert spherical → camera position

$$\begin{aligned} x &= r \cos \phi \sin \theta \\ y &= r \sin \phi \\ z &= r \cos \phi \cos \theta \end{aligned}$$

Mini-example:

```
camX = radius * Math.cos(phi) * Math.sin(theta);
```

```
camY = radius * Math.sin(phi);
camZ = radius * Math.cos(phi) * Math.cos(theta);
```

Then use the formulas above to build the view matrix.

MOUSE OPERATIONS — Orbit Camera

These use pure JavaScript events but directly affect the camera's view matrix.

Track mouse press

```
canvas.onmousedown = e => {
  dragging = true;
  lastX = e.clientX;
  lastY = e.clientY;
};
```

Mouse movement → update angles

```
if (dragging) {
  theta += (e.clientX - lastX) * 0.005;
  phi -= (e.clientY - lastY) * 0.005;
}
```

Mouse wheel → zoom

```
canvas.onwheel = e => {
  radius *= 1 + e.deltaY * 0.001;
};
```

Convert spherical → cartesian (camera position)

```
camX = radius * Math.cos(phi) * Math.sin(theta);
camY = radius * Math.sin(phi);
camZ = radius * Math.cos(phi) * Math.cos(theta);
```

Build view matrix each frame

```
const view = ____.lookAt([camX,camY,camZ], [0,0,0], [0,1,0]);
```

5. LOADING & PARSING OBJ MODELS (main.js + objLoader.js)

OBJ files contain:

- vertex positions (v x y z)
- texture coordinates (vt u v)
- normals (vn nx ny nz)
- faces (f v/t/n v/t/n v/t/n)

WebGL cannot read .obj files directly, so you must:

1. Fetch the OBJ file as text (in main.js)
2. Parse the text into arrays (in objLoader.js)
3. Upload arrays to WebGL buffers
4. Render using `gl.drawElements`

This section explains exactly how each part works.

A) loading the OBJ file

1. Fetch model text

```
const objText = await fetch("my_object.obj") then (res => res.text());
```

This loads the .obj file from the server as a plain string.

2. Parse OBJ text → usable vertex data

mesh.positions → [x,y,z, x,y,z, ...]

mesh.texcoords → [u,v, u,v, ...]

mesh.normals → [nx,ny,nz, ...]

mesh.indices → [i0,i1,i2, ...]

These arrays are now suitable for WebGL buffer uploads.

3. Upload parsed data to buffers (short form)

```
gl.bindBuffer(gl.ARRAY_BUFFER, posBuffer);
```

```
gl.bufferData(gl.ARRAY_BUFFER, new Float32Array(mesh.positions), gl.STATIC_DRAW);
```

```
gl.bindBuffer(gl.ARRAY_BUFFER, uvBuffer);
```

```
gl.bufferData(gl.ARRAY_BUFFER, new Float32Array(mesh.texcoords), gl.STATIC_DRAW);
```

```
gl.bindBuffer(gl.ARRAY_BUFFER, normalBuffer);
```

```
gl.bufferData(gl.ARRAY_BUFFER, new Float32Array(mesh.normals), gl.STATIC_DRAW);
```

```
gl.bindBuffer(gl.ELEMENT_ARRAY_BUFFER, indexBuffer);
```

```
gl.bufferData(gl.ELEMENT_ARRAY_BUFFER, new Uint32Array(mesh.indices), gl.STATIC_DRAW);
```

Now the OBJ model lives on the GPU.

4. Drawing the model (minimal)

```
gl.bindVertexArray(catVAO);
```

```
gl.drawElements(gl.TRIANGLES, mesh.indices.length, gl.UNSIGNED_INT, 0);
```

B) how OBJ text is parsed

This file contains one main function:

```
parseOBJ(text)
```

It reads the OBJ file line by line and interprets tokens.

1. Temporary storage (raw OBJ arrays)

Before parsing faces:

- v lines → push to tempPositions
- vt lines → push to tempTexcoords
- vn lines → push to tempNormals

Example:

```
v 1.0 0.5 -2.0
```

```
vt 0.3 0.8
```

```
vn 0.0 1.0 0.0
```

These are stored exactly as OBJ specifies.

2. Face parsing (f lines)

Example OBJ face:

```
f 1/2/3 4/5/6 7/8/9
```

Each group v/t/n refers to indices in the temporary arrays.

The loader:

1. Splits each face into components
2. Extracts the referenced v, vt, vn

3. Copies them into the final flattened arrays that WebGL wants
4. Generates indices for drawing

6. HOW OBJ PARSING WORKS

This section explains how the following parser works and what each part is responsible for:

```
function parseOBJ(text) { ... }
```

The parser's job is to take a .obj text file and convert it into flat WebGL arrays:

- positions → [x,y,z,x,y,z,...]
- texcoords → [u,v,u,v,...]
- normals → [nx,ny,nz,...]
- indices → [i0,i1,i2,...]

WebGL requires these arrays to draw 3D models.

A) Temporary Arrays — Matching the OBJ Format

```
const tempPos = [[0,0,0]];
const tempTex = [[0,0]];
const tempNor = [[0,0,0]];
```

OBJ uses 1-based indexing, so index 1 refers to the first vertex.

B) Final Arrays — Flattened for WebGL

```
const positions = [];
const texcoords = [];
const normals = [];
const indices = [];
```

These arrays are filled with expanded vertex data.

WebGL requires:

One vertex = one position + one uv + one normal.

But OBJ allows:

- same position with different UVs
 - same UV with different normals
- so we may need to duplicate vertices.
-

C) Vertex Deduplication Map

```
const vertMap = new Map();
```

OBJ face entries look like:

```
f 1/4/2 7/9/3 2/5/2
```

The string "1/4/2" uniquely identifies the combination:

- position index = 1
- texcoord index = 4
- normal index = 2

WebGL cannot reuse "partial" vertices, so:

If the same combination appears again, we reuse the already-created vertex index.

This avoids unnecessary duplication.

D) addVertex(key)

```
function addVertex(key) {
  if (vertMap.has(key)) return vertMap.get(key);
```

If we have seen this v/t/n combination before, reuse the index.

1. Split the v/t/n string

```
const parts = key.split('/');
const vIdx = parseInt(parts[0], 10);
const tIdx = parts[1] ? parseInt(parts[1], 10) : 0;
const nIdx = parts[2] ? parseInt(parts[2], 10) : 0;
OBJ format options:
```

```
v
v/vt
v//vn
v/vt/vn
```

So we must check for missing parts.

2. Fetch referenced arrays

```
const v = tempPos[vIdx];
const t = tempTex[tIdx] || [0,0];
const n = tempNor[nIdx] || [0,1,0];
```

- If texcoord is missing → use [0,0]
- If normal is missing → use [0,1,0] as fallback

3. Push flattened vertex attributes

```
positions.push(v[0], v[1], v[2]);
texcoords.push(t[0], 1.0 - t[1]); // flip V axis
normals.push(n[0], n[1], n[2]);
```

Note: WebGL textures have origin at top-left; OBJ uses bottom-left → flip V.

4. Store new index and return it

```
const idx = positions.length / 3 - 1;
vertMap.set(key, idx);
return idx;
```

This is the vertex index used in indices[].

E) Face Parsing — Triangulation of Polygons

OBJ face line:

f a b c d ...

Could be triangles or quads.

This loader converts any polygon into triangles using triangle fan:

```
for (let i = 1; i < faceVerts.length - 1; i++) {
    indices.push(addVertex(faceVerts[0]),
                  addVertex(faceVerts[i]),
                  addVertex(faceVerts[i+1]));
}
```

This produces WebGL-friendly index buffers.

F) Final Returned Object

```
return { positions, texcoords, normals, indices };
```

These arrays are ready for:

- `gl.bindBuffer(gl.ARRAY_BUFFER, ...)`
 - `gl.bufferData(...)`
 - `gl.vertexAttribPointer(...)`
 - `gl.drawElements(gl.TRIANGLES, ...)`
-